



Estándar de codificación Web

Sistema de Control de Acceso para el Campus Universitario

Valadez Carmona Guadalupe Yamileth
Rodríguez Cervantes Kevin Manzur
Viveros Hernández Alejandro

Administración de Proyectos 1



Tlamati Access

07-marzo-2026

DV-1.1.0



Control de versiones

Versión	Fecha	Autor	Cambios realizados	Aprobado
DV-1.0.0	2026-03-07	Manzur	Versión inicial del documento Estándar de codificación Web	Yamileth Manzur
DV-1.1.0	2026-03-08	Manzur	Selección del lenguaje de programación para el desarrollo del proyecto. Conforme al código, establece el formato de comentarios de errores y documentación.	Yamileth Manzur Alejandro

CONTENIDO

1. Introducción.....	1
2. Documentación en php (laravel)	1
2.1. Estructura Base de PHPDoc.....	1
2.2. Documentación de Clases	2
2.3. Documentación de Métodos y Funciones	2
2.4. Documentación de Propiedades y Atributos.....	3
2.5. Comentarios Inline y de Bloque	4
2.6. Comentarios a Evitar	4
2.7. Documentación de Controladores (Laravel)	5
3. Documentación en javascript	6
3.1. Estándar JSDoc para Funciones	6
3.2. Documentación de Clases y Componentes (Vue.js)	7
3.3. Comentarios para Lógica Compleja	7
3.4. Documentación de Módulos y Utilidades	8
4. Documentación en CSS/SCSS	9
4.1. Comentarios de Sección y Organización	9
4.2. Convenciones de Nomenclatura Documentada	11
5. Tipos de comentarios y su uso apropiado	11
5.1. Matriz de Decisiones para Comentarios.....	11
5.2. Estándar para Comentarios de "Para Otro Programador"	12
6. Checklist de revisión de código.....	13
6.1. Antes de Enviar un Pull Request	13
7. Mantenimiento y actualización de documentación	13

1. Introducción

Este estándar establece las convenciones obligatorias para la documentación de código en el proyecto. Su objetivo es garantizar:

- Código comprensible para cualquier desarrollador del equipo
- Facilitar modificaciones y correcciones futuras
- Permitir trabajo paralelo sin conflictos de interpretación
- Generar documentación automática para APIs y módulos
- Acelerar la incorporación de nuevos desarrolladores

Aplica a todo el código fuente del proyecto:

- Backend: PHP (Laravel: Controllers, Models, Services, Jobs, etc.)
- Frontend: JavaScript (Vue.js/Alpine.js, scripts personalizados)
- Estilos: CSS/SCSS (archivos de estilo y componentes)
- Configuración: Archivos de configuración documentados cuando sea necesario

2. Documentación en php (laravel)

2.1. Estructura Base de PHPDoc

Todo elemento público/protegido debe incluir bloque PHPDoc con esta estructura:

```
/**
 * [Título breve y descriptivo en una línea]
 *
 * [Descripción detallada opcional. Explica el propósito,
 * contexto o decisiones de diseño importantes.]
 *
 * @param TipoParam $nombreParam [Descripción del parámetro]
 * @param TipoParam2 $otroParam [Descripción]
 * @return TipoRetorno [Descripción del valor retornado]
 *
 * @throws ExceptionType [Cuándo y por qué se lanza]
 * @author [Nombre] [Fecha - opcional para cambios críticos]
 * @see [Referencia a clase/método relacionado]
 * @deprecated [Versión] [Razón y alternativa si aplica]
 */
```



2.2. Documentación de Clases

```
<?php

namespace App\Services;

use App\Models\User;
use Illuminate\Support\Facades\Log;

/**
 * Servicio para gestión de credenciales estudiantiles.
 *
 * Este servicio centraliza la lógica de generación, validación
 * y almacenamiento de credenciales de acceso para el sistema
 * de control escolar. Implementa patrón Service para separar
 * responsabilidades del controlador.
 *
 * @package App\Services
 * @category Security
 * @version 1.0
 */
class CredentialService
{
    // ...
}
```

Elementos requeridos:

Etiqueta	Obligatoria	Descripción
Descripción breve	Si	Resumen en una línea
Descripción larga	Medio	Cuando la lógica es compleja
@package	Si	Namespace principal
@category	Medio	Para clasificación en docs generados
@version	Medio	En clases públicas/API

2.3. Documentación de Métodos y Funciones

```
/**
 * Genera credencial física con fotografía sintética.
 *
 * Procesa los datos del estudiante, valida la imagen asignada
 * y genera el archivo PDF listo para impresión en formato
 * CR80 estándar (85.60 × 53.98 mm).
 *
 * @param \App\Models\Student $student Instancia del estudiante registrado
 * @param string $photoPath Ruta relativa de la imagen en storage/app
 * @param array $options Configuración adicional:
```



```

*         - 'print_ready': bool (default: true)
*         - 'include_qr': bool (default: true)
*         - 'template': string (nombre de plantilla)
* @return \Illuminate\Http\Response PDF binario para descarga
*
* @throws \Illuminate\Database\Eloquent\ModelNotFoundException Si el estudiante no existe
* @throws \Symfony\Component\HttpFoundation\Exception\BadRequestException Si la imagen es inválida
*
* @see \App\Http\Controllers\CredentialController@download
* @see config('credentials.print_settings')
*/
public function generatePhysicalCredential(
    Student $student,
    string $photoPath,
    array $options = []
): Response {
    // Implementación...
}

```

Reglas para parámetros complejos:

```

/**
 * @param array $filters Criterios de filtrado:
 *         - 'status': 'active'|'inactive'|'all' (default: 'active')
 *         - 'date_from': string (Y-m-d) Fecha inicio
 *         - 'date_to': string (Y-m-d) Fecha fin
 *         - 'group_by': 'department'|'role'|null
 * @param int $perPage Registros por página (10-100, default: 25)
 * @param string $sort Campo para ordenamiento con dirección: 'name:asc', 'created_at:desc'
 */

```

2.4. Documentación de Propiedades y Atributos

```

class Student extends Model
{
    /**
     * Los atributos que son asignables masivamente.
     */
    * @var array<int, string>
    */
    protected $fillable = [
        'enrollment_id',
        'full_name',
        'email',
        'photo_path',
        'status'
    ];
}
/**

```



```
* Los atributos que deben ser ocultados en arrays/JSON.
*
* @var array<int, string>
*/
protected $hidden = [
    'password',
    'remember_token',
    'internal_notes'
];

/**
 * Los atributos que deben ser convertidos a tipos nativos.
 *
 * @var array<string, string>
 */
protected $casts = [
    'email_verified_at' => 'datetime',
    'is_active' => 'boolean',
    'metadata' => 'array'
];
}
```

2.5. Comentarios Inline y de Bloque

Comentarios de Implementación (CUÁNDO usarlos)

```
// * Explicar lógica no obvia
// Usamos soft deletes para mantener historial de credenciales
// sin eliminar registros que podrían tener relaciones críticas
$this->withTrashed();

// * Documentar decisiones de negocio
// El límite de 3 intentos sigue la política de seguridad
// institucional POL-SEC-2024-003
$maxAttempts = 3;

// * Referenciar bugs o issues
// FIXME: Optimizar consulta para >10k registros (Issue #247)
// TODO: Implementar caché Redis para respuestas frecuentes
// NOTE: Esta validación se mantiene por compatibilidad con API v1
```

2.6. Comentarios a Evitar

```
// ** Redundante (el código ya lo dice)
// Incrementa el contador en 1
$count++;
```



```
// ** Demasiado obvio
// Si el usuario está activo
if ($user->is_active) {
    // Haz algo
    doSomething();
}

// ** Comentarios desactualizados
// Retorna un array (ahora retorna Collection)
```

2.7. Documentación de Controladores (Laravel)

```
<?php

namespace App\Http\Controllers;

use App\Http\Requests\StoreCredentialRequest;
use App\Services\CredentialService;
use Illuminate\Http\JsonResponse;

/**
 * Controlador para operaciones de credenciales de acceso.
 *
 * Gestiona el ciclo completo de credenciales: creación,
 * visualización, actualización y revocación.
 *
 * @middleware auth:sanctum, role:admin|coordinator
 * @resource Credentials
 */
class CredentialController extends Controller
{
    /**
     * Display a listing of active credentials.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\JsonResponse Collection paginada de credenciales
     *
     * @OA\Get(
     *     path="/api/v1/credentials",
     *     summary="Listar credenciales activas",
     *     tags={"Credentials"},
     *     security={{"sanctum":{}}},
     *     @OA\Response(
     *         response=200,
     *         description="Lista exitosa",
     *         @OA\JsonContent(ref="#/components/schemas/CredentialCollection")
     *     )
     * )
     */
}
```




```

public function index(Request $request): JsonResponse
{
    // ...
}

/**
 * Store a newly created credential in storage.
 *
 * @param \App\Http\Requests\StoreCredentialRequest $request
 * @param \App\Services\CredentialService $service
 * @return \Illuminate\Http\JsonResponse Recurso creado con código 201
 *
 * @throws \Illuminate\Validation\ValidationException Si la validación falla
 */
public function store(StoreCredentialRequest $request, CredentialService $service): JsonResponse
{
    // ...
}
}

```

3. Documentación en javascript

3.1. Estándar JSDoc para Funciones

```

/**
 * Valida y formatea fotografía para credencial estudiantil.
 *
 * Procesa la imagen en el cliente: verifica dimensiones,
 * formato y tamaño antes del upload al servidor.
 *
 * @param {File} imageFile - Archivo de imagen seleccionado por el usuario
 * @param {Object} options - Configuración de validación
 * @param {number} [options.maxSizeMB=2] - Tamaño máximo en megabytes
 * @param {number[]} [options.allowedDimensions=[400,400]] - Dimensiones permitidas [ancho, alto]
 * @param {string[]} [options.acceptedFormats=['image/jpeg', 'image/png']] - MIME types aceptados
 * @returns {Promise<{valid: boolean, message?: string, processedFile?: File}>}
 *   Objeto con resultado de validación y archivo procesado si es válido
 * @throws {TypeError} Si imageFile no es instancia de File
 *
 * @example
 * const result = await validateStudentPhoto(selectedFile, { maxSizeMB: 1.5 });
 * if (result.valid) {
 *   uploadToServer(result.processedFile);
 * }
 */
export async function validateStudentPhoto(imageFile, options = {}) {
    // Implementación...
}

```



}

3.2. Documentación de Clases y Componentes (Vue.js)

```

/**
 * Componente para vista previa de credencial en tiempo real.
 *
 * Muestra una representación visual de la credencial con los
 * datos del estudiante, actualizándose reactivamente conforme
 * se completan los campos del formulario.
 *
 * @component CredentialPreview
 * @props {Object} student - Datos del estudiante (required)
 * @props {String} student.fullName - Nombre completo para mostrar
 * @props {String} [student.photoUrl] - URL de la fotografía (opcional)
 * @props {String} student.enrollmentId - Matrícula institucional
 * @props {Boolean} [showQR=true] - Mostrar código QR en la vista previa
 * @emits CredentialPreview#photo-click - Cuando se hace clic en la fotografía
 * @emits CredentialPreview#edit-request - Cuando se solicita editar datos
 *
 * @slot header - Contenido personalizado para encabezado de credencial
 * @slot footer - Contenido personalizado para pie de credencial
 *
 * @example
 * <CredentialPreview
 *   :student="formData"
 *   @photo-click="openPhotoSelector"
 * />
 */
export default {
  name: 'CredentialPreview',
  props: {
    student: {
      type: Object,
      required: true,
      validator: (value) => {
        return value.fullName && value.enrollmentId;
      }
    },
    // ... más props documentadas individualmente
  },
  // ...
}

```

3.3. Comentarios para Lógica Compleja

```
// * Documentar algoritmos no triviales
```



© 2026 Universidad Autónoma de la Ciudad de México
Proyecto Tlamati Access - Todos los derechos reservados

Versión documento: 1.1.0



```

/**
 * Calcula posición óptima para código QR en credencial.
 *
 * Usa algoritmo de empaquetamiento binario para asegurar que
 * el QR no sobrescriba texto crítico, considerando márgenes
 * de seguridad de impresión (3mm mínimo desde bordes).
 *
 * @private
 * @param {Object} dimensions - Dimensiones de la credencial en mm
 * @param {Object} qrSize - Dimensiones del código QR
 * @returns {Object} Coordenadas {x, y} para posicionamiento
 */
_calculateQRPosition(dimensions, qrSize) {
  // Lógica de cálculo...
}

// * Explicar workarounds o hacks necesarios
// WORKAROUND: Safari no soporta URL.createObjectURL para blobs
// en iframes de impresión. Usamos FileReader como fallback.
// Ver: https://bugs.webkit.org/show_bug.cgi?id=XXXXX
if (this._isSafari() && this._isPrinting()) {
  // Implementación alternativa...
}

// * Referenciar documentación externa
// Según especificación CR80 ISO/IEC 7810:2019
// https://www.iso.org/standard/58887.html
const CREDENTIAL_WIDTH_MM = 85.60;
const CREDENTIAL_HEIGHT_MM = 53.98;

```

3.4. Documentación de Módulos y Utilidades

```

/**
 * Módulo de utilidades para manejo de credenciales.
 * @module utils/credentials
 */

/**
 * Convierte dimensiones de mm a píxeles para impresión.
 * @function mmToPx
 * @param {number} mm - Valor en milímetros
 * @param {number} [dpi=300] - Resolución de impresión (puntos por pulgada)
 * @returns {number} Valor equivalente en píxeles
 */
export const mmToPx = (mm, dpi = 300) => {
  return Math.round(mm * dpi / 25.4);
};

/**
 * Genera identificador único para transacciones de impresión.

```



```

* @function generatePrintTransactionId
* @returns {string} ID con formato: PRT-YYYYMMDD-XXXXXX
*/
export const generatePrintTransactionId = () => {
  const date = new Date().toISOString().slice(0, 8).replace(/-/g, "");
  const random = Math.random().toString(36).substring(2, 8).toUpperCase();
  return `PRT-${date}-${random}`;
};

```

4. Documentación en CSS/SCSS

4.1. Comentarios de Sección y Organización

```

// =====
// CREDENTIAL CARD COMPONENT
// =====
// Estilos para la representación visual de credenciales estudiantiles
// Framework: BEM methodology
// Breakpoints: mobile-first
// =====

// -----
// Variables de diseño
// -----
// Dimensiones estándar CR80 (ISO/IEC 7810)
$credential-width: 85.60mm;
$credential-height: 53.98mm;
$credential-border-radius: 3.5mm;

// Colores institucionales
$color-primary: #003366; // Azul institucional
$color-secondary: #CCCCCC; // Gris para bordes
$color-text: #1A1A1A; // Texto principal

// -----
// Mixins reutilizables
// -----

/**
 * Aplica estilos de impresión optimizados
 * @param {Boolean} $include-bleed - Incluir margen de sangrado (default: true)
 * @param {String} $orientation - Orientación: 'portrait' | 'landscape'
 */
@mixin print-optimized($include-bleed: true, $orientation: 'portrait') {
  @media print {
    // Configuración de impresión...
  }
}

```



```

}

// -----
// Estructura base de credencial (BEM)
// -----

/**
 * @block credential-card
 * Contenedor principal de la credencial visual
 */
.credential-card {
  width: $credential-width;
  height: $credential-height;
  border-radius: $credential-border-radius;
  // ...

  /**
   * @element credential-card__header
   * Encabezado con logo y nombre institucional
   */
  &__header {
    // ...
  }

  /**
   * @element credential-card__photo
   * Contenedor de fotografía del estudiante
   * @modifier credential-card__photo--placeholder - Muestra avatar por defecto
   */
  &__photo {
    // ...

    &--placeholder {
      background-color: $color-secondary;
    }
  }

  /**
   * @element credential-card__details
   * Sección de información del estudiante
   */
  &__details {
    // ...
  }
}

// -----
// Estados y variantes
// -----

/**
 * @modifier credential-card--print-ready
 * Ajustes específicos para versión de impresión

```



```

*/
.credential-card--print-ready {
  @include print-optimized();
}

/**
 * @modifier credential-card--preview
 * Estilos para vista previa en pantalla (no impresión)
 */
.credential-card--preview {
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.15);
  // ...
}

```

4.2. Convenciones de Nomenclatura Documentada

```

// * Patrones de nombres documentados

// Componentes: kebab-case con prefijo de módulo
.credential-form__input-group { }
.credential-form__error-message { }

// Estados: prefijo is- o has-
.is-printing { }
.has-photo { }
.is-invalid { }

// Utilidades: prefijo u- seguido de propiedad
.u-text-center { }
.u-mb-2 { } // margin-bottom: 0.5rem
.u-w-full { }

// JavaScript hooks: prefijo js- (NO usar para estilos)
// Estos selectores son SOLO para JS, no aplicar estilos
.credential-card [data-js-photo-preview] { }
.form-submit [data-js-submit-btn] { }

```

5. Tipos de comentarios y su uso apropiado

5.1. Matriz de Decisiones para Comentarios

Tipo de Comentario	Sintaxis	Cuándo Usar	Ejemplo
	© 2026 Universidad Autónoma de la Ciudad de México Proyecto Tlamati Access - Todos los derechos reservados	Versión documento: 1.1.0	

PHPDoc/JSDoc Block	<code>/** */</code>	Funciones, clases, métodos públicos	Documentación de API
Comentario de Línea	<code>// (PHP/JS)</code>	Explicar lógica específica inline	<code>// Validar formato antes de guardar</code>
Comentario de Bloque	<code>/* */ (CSS)</code>	Secciones grandes de CSS	<code>/* ===== CONFIGURACIÓN DE IMPRESIÓN ===== */</code>
TODO/FIXME/NOTE	<code>// TODO:</code>	Tareas pendientes o advertencias	<code>// TODO: Implementar caché para consultas frecuentes</code>
Deprecated	<code>@deprecated</code>	Marcar código obsoleto	<code>@deprecated 2.0 Usar generateCredentialV2()</code>

5.2. Estándar para Comentarios de "Para Otro Programador"

```

/**
 * ---+ NOTA PARA DESARROLLADORES +---
 *
 * Este método implementa un workaround para un bug conocido en
 * Laravel 10.48 con relaciones polimórficas y soft deletes.
 *
 * PROBLEMA: Cuando se consulta un Student con credentials eliminadas,
 *           Laravel no aplica correctamente el scope global.
 *
 * SOLUCIÓN: Usamos withTrashed() explícito + filtro manual en colección.
 *
 * REFERENCIA:
 * - Issue Laravel: https://github.com/laravel/framework/issues/XXXXX
 * - Ticket interno: DEV-2026-089
 *
 * REVISAR: Eliminar este workaround al actualizar a Laravel 11+
 *
 * @see \App\Models\Student::credentials()
 */
public function getActiveCredentialsWithHistory()
{
    // Implementación con workaround...
}

```

Formato estandarizado para notas críticas:

```

---+ [TIPO]: [Título breve]

[Descripción clara del contexto o problema]

CAUSA: [Explicación técnica si aplica]
SOLUCIÓN: [Qué se hizo y por qué]
REFERENCIA: [Links a issues, documentación, tickets]
REVISAR: [Cuándo o bajo qué condiciones se puede eliminar]

```



6. Checklist de revisión de código

6.1. Antes de Enviar un Pull Request

* Checklist de Documentación

PHP

- [] Todas las clases públicas tienen PHPDoc con descripción breve
- [] Métodos públicos/protegidos documentan parámetros y retorno
- [] Excepciones lanzadas están declaradas con @throws
- [] Comentarios inline explican "por qué", no "qué"
- [] No hay comentarios redundantes o desactualizados
- [] Código complejo tiene explicación de algoritmo/decisión

JavaScript

- [] Funciones exportadas tienen JSDoc completo
- [] Componentes Vue documentan props, eventos y slots
- [] Tipos de parámetros y retorno especificados
- [] Ejemplos de uso en funciones complejas
- [] Comentarios para workarounds o lógica no obvia

CSS/SCSS

- [] Secciones principales tienen comentarios de cabecera
- [] Clases BEM documentadas con @block/@element/@modifier
- [] Variables con descripción de propósito y valores
- [] Mixins/function con parámetros y retorno documentados

General

- [] No hay TODO/FIXME sin ticket asociado
- [] Código deprecated marcado con @deprecated y alternativa
- [] Referencias externas (issues, docs) con URLs completas
- [] Documentación en inglés técnico consistente

7. Mantenimiento y actualización de documentación

REGLA: "La documentación se actualiza en el mismo commit que el código"

Hacer:

- Modificar PHPDoc/JSDoc al cambiar firma de método
- Actualizar comentarios al refactorizar lógica compleja
- Agregar @deprecated antes de eliminar funcionalidad



© 2026 Universidad Autónoma de la Ciudad de México
Proyecto Tlamati Access - Todos los derechos reservados

Versión documento: 1.1.0



- Revisar documentación en cada sprint planning

No hacer:

- Dejar "arreglar documentación después" como tarea separada
- Actualizar código sin revisar docs relacionados
- Ignorar warnings de herramientas de análisis estático

